

Topological Data Analysis for Time Series Classification

Background and Feature Design

İsmail Güzel

May 2026

Contents

1	Introduction	2
2	Persistent Homology	3
2.1	Intuition: the rising-water analogy	3
2.2	Simplicial Complexes and Homology	3
2.3	Filtrations	3
2.4	Persistence Diagrams	4
3	Filtrations for Univariate Time Series	4
3.1	Sublevel Set Filtration (sub_H0)	4
3.2	Superlevel Set Filtration (sup_H0)	5
3.3	Takens Embedding and Vietoris–Rips Filtration (H1)	5
4	Scalar Feature Extraction from Persistence Diagrams	6
4.1	Carlsson Coordinates	6
4.2	Persistent Entropy	7
4.3	Persistence Landscapes	7
4.4	Summary: 8 Features per Diagram	8
5	Feature Selection	9
6	Literature Review: TDA for Time Series	9
6.1	Foundations	9
6.2	Vectorisation Methods	10
6.3	TDA Applied to Time Series	10
6.4	TDA and Machine Learning	11
7	Empirical Performance Summary	11
8	Conclusion	14

1 Introduction

Motivation: what shape does a time series have?

Consider a time series that records, say, the hourly temperature over a month. A statistician would summarise it with its mean, variance, autocorrelation coefficients, and spectral density. These are all perfectly reasonable — but they are all *coordinate-dependent*: they depend on the absolute scale and the temporal ordering of the values.

Now ask a different question: if you drew the temperature curve on paper, what would you notice about its *shape*? You might say: “it goes up and down periodically”, “there is a persistent plateau around day 10”, “there is one very sharp spike on day 22”. These are *topological* observations — they describe how the curve connects, bends, and self-organises, independently of the exact numerical values.

Topological Data Analysis (TDA) is the branch of applied mathematics that makes these observations precise and computable (Carlsson, 2009; Edelsbrunner & Harer, 2010). The key idea is to *vary a threshold* and track how connected pieces of the data are born, merge, and disappear. The result is a concise multi-scale summary called a *persistence diagram*.

Why persistence diagrams are useful for time series

The central object of TDA is the *persistence diagram*, a compact summary of how topological features are born and die as a filtration threshold is varied. Three properties make it particularly useful in practice:

1. **Stability.** Persistence diagrams are stable under small perturbations: adding a little noise to a time series shifts the diagram by at most the same amount in the bottleneck distance (Cohen-Steiner et al., 2007). This is the topological analog of Lipschitz continuity.
2. **Scale invariance.** The diagram captures structure that is present across *multiple scales* simultaneously. A trend and a superimposed oscillation both leave traces in the same diagram.
3. **Complementarity.** Persistence diagrams encode global shape rather than local statistics. They are therefore complementary to statistical feature extraction methods such as TSFresh (Christ et al., 2018), which excel at local temporal patterns, autocorrelation structure, and spectral content.

This complementarity motivates the hybrid pipeline studied in this work: combining 18 topological features with 100 TSFresh features for the classification of synthetic time series patterns.

Structure of this report

Section 2 introduces persistent homology, the mathematical foundation, with an intuitive water-level analogy before the formal definitions. Section 3 describes the three filtration strategies applied to univariate time series. Section 4 details the scalar features derived from each persistence diagram. Section 5 explains the mutual information-based selection procedure used to reduce the feature set. Section 6

reviews relevant literature, from foundational papers to recent (2022–2026) time series applications. Section 7 summarizes the empirical performance of the topological features on the 10-class classification benchmark used in this work.

2 Persistent Homology

2.1 Intuition: the rising-water analogy

Before introducing the formal machinery, it helps to build intuition with a concrete analogy.

Imagine a hilly landscape and a rising sea level. As the water rises:

- First, the highest peaks emerge as isolated *islands* (connected components are born).
- As the water rises further, nearby islands merge into *archipelagos* (components die by merging with older ones).
- Enclosed *lakes* may appear and then fill in (loops are born and die).

The *birth height* of an island is the elevation of its highest peak; its *death height* is the elevation at which it merges with a taller island. The difference — the *persistence* — measures how prominent the peak is. A tall mountain has high persistence; a small bump has low persistence.

For a time series, the “landscape” is the graph of the signal values. The “water level” is a varying threshold. As we raise (or lower) this threshold, connected groups of time steps appear and merge, and the birth/death pairs encode the structural features of the signal.

2.2 Simplicial Complexes and Homology

Let X be a finite point cloud or a function defined on a topological space. Algebraic topology studies X through the lens of *homology groups* $H_k(X; \mathbb{F})$ over a field $\mathbb{F}$, where the integer $k \geq 0$ is the *homological dimension*:

- H_0 : connected components (*islands* in the water analogy)
- H_1 : one-dimensional loops (*tunnels* through the terrain)
- H_2 : two-dimensional voids (*cavities* enclosed in 3D)

In practice, X is replaced by a *simplicial complex* built from the data, and homology is computed combinatorially via chain complexes and boundary operators (Hatcher, 2002). For a 1-dimensional time series the simplicial complex is simply a graph whose nodes are the time steps and whose edges connect consecutive steps.

2.3 Filtrations

A *filtration* is a nested sequence of simplicial complexes

$$\emptyset = K_0 \subseteq K_1 \subseteq \cdots \subseteq K_n = K$$

parameterised by a threshold $t_0 \leq t_1 \leq \cdots \leq t_n$. As the threshold increases, new simplices are added and the topology changes: connected components merge, loops appear or fill in.

Definition 2.1 (Birth and Death). A topological feature (e.g., a connected component) is *born* at threshold b when it first appears in the filtration and *dies* at threshold $d > b$ when it merges with an older feature or is filled by a higher-dimensional simplex. Its *persistence* is $\text{pers} = d - b$.

2.4 Persistence Diagrams

The *persistence diagram* $\text{Dgm}_k(f)$ for homological dimension k is the multiset of points $(b_i, d_i) \in \mathbb{R}^2$ corresponding to all features that are born and die during the filtration, together with the diagonal $\{(t, t) : t \in \mathbb{R}\}$ counted with infinite multiplicity (Edelsbrunner et al., 2002).

Features near the diagonal (small persistence $d - b$) are considered topological noise; features far from the diagonal encode robust structural properties of the signal.

A fundamental stability result guarantees that small perturbations of the input function produce small changes in the persistence diagram under the bottleneck distance d_B (Cohen-Steiner et al., 2007):

Theorem 2.1 (Stability of Persistence Diagrams). *Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a compact topological space X . Then*

$$d_B(\text{Dgm}(f), \text{Dgm}(g)) \leq \|f - g\|_\infty.$$

This stability property is crucial for applications to noisy time series: small measurement errors cannot drastically alter the topological summary.

3 Filtrations for Univariate Time Series

Given a univariate time series $f : \{0, 1, \dots, T - 1\} \rightarrow \mathbb{R}$, we apply three complementary filtration strategies. Together they form the `sublevel+h1` pipeline used in this work.

3.1 Sublevel Set Filtration (sub_H0)

Definition 3.1 (Sublevel Set). The *sublevel set* of f at threshold t is

$$\mathcal{L}^-(t) = \{s : f(s) \leq t\}.$$

As t increases from $\min f$ to $\max f$, time steps are added to the active set in increasing order of their function value. Two active time steps s_1, s_2 are connected if they are adjacent in time. Connected components are born when isolated time steps are added and die when they merge with an older component (by the elder rule).

The resulting H_0 persistence diagram encodes how the *local minima* of f are organised. A time series with a single dominant minimum (e.g., a deterministic trend reaching its nadir at one point) produces a diagram with one highly persistent point far from the diagonal, while a noisy stationary process produces many low-persistence points clustered near the diagonal (noise). The sublevel filtration is equivalent to computing the zeroth persistent homology of the 1-dimensional CW complex built on the graph of f .

Example 3.1 (Sublevel filtration on a mean-shift series). Consider a time series f with a mean of 0 for $t < 500$ and a mean of 2 for $t \geq 500$ (a mean shift). As the threshold t rises from $-\infty$:

- i. Near $t = 0$: the segment around the first half is activated; one connected component is born.
- ii. Near $t = 2$: the segment around the second half is activated; a second component is born.
- iii. At the threshold where the two segments touch (at the shift boundary): the younger component merges into the older one — it dies.

The resulting persistence diagram contains a point at approximately $(2, 2+\delta)$ — i.e., it has low persistence — while the first component persists until $t = \max f$, giving a high-persistence point. This asymmetric diagram is the topological signature of a mean shift.

3.2 Superlevel Set Filtration (sup_H0)

Definition 3.2 (Superlevel Set). The *superlevel set* of f at threshold t is

$$\mathcal{L}^+(t) = \{s : f(s) \geq t\}.$$

As t decreases from $\max f$ to $\min f$, time steps are added to the active set in decreasing order of their function value. This filtration captures the organisation of *local maxima*: a sharp spike (point anomaly) creates a single highly persistent component because the spike maximum appears early and persists until the threshold drops to the baseline level. Mean shifts create step changes in the superlevel diagram. The superlevel diagram is the mirror image of the sublevel diagram when f is replaced by $-f$.

3.3 Takens Embedding and Vietoris–Rips Filtration (H1)

The sublevel and superlevel filtrations only capture H_0 features (connected components). To access H_1 features (loops), we apply a different approach based on *delay embedding* (Takens, 1981).

Definition 3.3 (Takens Delay Embedding). Given a time series f of length T , a delay $\tau > 0$, and an embedding dimension d , the *Takens embedding* maps each time step s to the point

$$\phi(s) = (f(s), f(s + \tau), \dots, f(s + (d - 1)\tau)) \in \mathbb{R}^d.$$

By Takens’ embedding theorem (Takens, 1981), if f arises from a deterministic dynamical system, then ϕ reconstructs the attractor of the system up to diffeomorphism (under generic conditions). For a periodic or quasi-periodic signal, the attractor is a torus or circle, and H_1 detects the resulting loops.

Example 3.2 (Periodicity via H_1). A sinusoidal signal $f(t) = \sin(2\pi t/P)$ with period P , when delay-embedded with $\tau = P/4$ and $d = 2$, produces the point cloud $\{(\sin \theta, \cos \theta)\}$ — a perfect circle. Under Vietoris–Rips filtration, this circle creates exactly one persistent H_1 class born at small ϵ and dying only when the circle is “filled in” at large ϵ . Its persistence is proportional to the signal amplitude. A stationary noise process, by contrast, produces a point cloud without circular structure, giving only short-lived H_1 classes near the diagonal.

The point cloud $\{\phi(s)\}$ is filtered by the *Vietoris–Rips complex*: at radius ϵ , any set of points mutually within distance ϵ forms a simplex:

$$\mathcal{R}_\epsilon = \{\sigma \subseteq P : \text{diam}(\sigma) \leq \epsilon\}.$$

As ϵ increases from 0 to ∞ , H_1 classes (loops) are born and die; a persistent loop indicates genuine cyclic or oscillatory structure in the time series.

Important limitation. The sublevel and superlevel filtrations are constructed from the *values* of the time series and are therefore invariant to any permutation of time steps that preserves the multiset of values. They encode “what heights appear” but not “when they appear”. This means that stationarity, autocorrelation structure, and volatility clustering — properties that are inherently temporal — are not directly captured. The Takens embedding partially addresses this by introducing temporal ordering through the delay parameter τ , but the basic sublevel/superlevel features share this limitation. This is the primary reason why topological features underperform statistical features on the `stationary` class in the experiments described in Section 7.

Why three filtrations? Each filtration answers a different topological question about the time series:

Table 1: Complementary information captured by the three filtration strategies.

Diagram	Filtration	Dimension	Captures
sub_H0	Sublevel sets	H_0	Local minima, connectivity from below
sup_H0	Superlevel sets	H_0	Local maxima, connectivity from above
H1	Takens + Rips	H_1	Cyclic structure, delay-space loops

4 Scalar Feature Extraction from Persistence Diagrams

A persistence diagram is a *multiset* of points in $\mathbb{R}^2$ and cannot be directly fed into standard machine learning pipelines. Several vectorisation methods have been proposed to convert diagrams to fixed-length feature vectors. We use a combination of Carlsson coordinates, persistence entropy, and persistence landscapes.

4.1 Carlsson Coordinates

Adcock et al. (2016) proposed a set of polynomial functions on the space of persistence diagrams that are invariant under rearrangement of the points. Let $D = \{(b_i, d_i)\}$ be a persistence diagram (diagonal points excluded) and let $p_i = d_i - b_i$ denote the persistence of the i -th feature.

The five Carlsson-type invariants used in this work are:

$$f_1(D) = \sum_i b_i \cdot p_i \quad (1)$$

$$f_2(D) = \sum_i (d_{\max} - d_i) \cdot p_i \quad (2)$$

$$f_3(D) = \sum_i b_i^2 \cdot p_i^4 \quad (3)$$

$$f_4(D) = \sum_i (d_{\max} - d_i)^2 \cdot p_i^4 \quad (4)$$

$$f_5(D) = \max_i p_i \quad (5)$$

where $d_{\max} = \max_i d_i$. The first two coordinates are linear in persistence and use birth/death as weights — f_1 is the *birth-weighted total persistence* (a signed quantity that captures *where* on the filtration axis the bars are concentrated; it can be negative when births lie below zero), and f_2 is the analogous *death-distance-weighted total persistence*. The third and fourth coordinates raise both factors to higher powers (b_i^2 or $(d_{\max} - d_i)^2$ multiplied by p_i^4); the quartic in p_i aggressively down-weights low-persistence (noise) bars and emphasises long-lived (signal) features, while the quadratic birth/death weight sharpens the contrast between bars at extreme filtration levels and those near the centre. Finally, f_5 is simply the single most persistent bar — a robust summary of the dominant topological feature in the diagram. This particular five-statistic set follows the form used by Umeda (2017) for time-series classification rather than the original ring-of-polynomial-invariants construction of Adcock et al. (2016), which proposes a larger family of polynomial features.

4.2 Persistent Entropy

Persistent entropy (Chintakunta et al., 2015) adapts the Shannon entropy formula to measure the spread of persistence values in a diagram:

$$E(D) = - \sum_i \hat{p}_i \log \hat{p}_i, \quad \hat{p}_i = \frac{p_i}{\sum_j p_j}, \quad (6)$$

where the sum runs over all off-diagonal points. $E(D) = 0$ when the diagram has a single point of maximal persistence (completely concentrated signal), and $E(D)$ is maximal when all persistence values are equal (uniform diagram, typical of white noise). Persistent entropy thus functions as a complexity measure: low entropy indicates a dominant structural feature; high entropy indicates homogeneous noise.

4.3 Persistence Landscapes

The persistence landscape (Bubenik, 2015) is a functional representation of the persistence diagram that maps each point (b_i, d_i) to a tent function:

$$\lambda_i(t) = \begin{cases} t - b_i & \text{if } b_i \leq t \leq \frac{b_i + d_i}{2}, \\ d_i - t & \text{if } \frac{b_i + d_i}{2} \leq t \leq d_i, \\ 0 & \text{otherwise.} \end{cases}$$

The k -th landscape function $\lambda_k(t)$ is the k -th largest value among all $\lambda_i(t)$ at each threshold t . The landscape lives in a Banach space and admits a well-defined mean and variance over samples, making it amenable to statistical inference.

Discretisation used in this work. We discretise the filtration axis into $R = 100$ uniform grid points spanning $[\min_i b_i, \max_i d_i]$, and on each grid point we store the top- K landscape values $\lambda_1(t), \dots, \lambda_K(t)$ with $K = 5$. The five landscape levels are then averaged into a single discrete function

$$\bar{\lambda}(t) = \frac{1}{K} \sum_{k=1}^K \lambda_k(t), \quad (7)$$

which we treat as a vector $\bar{\lambda} \in \mathbb{R}^R$. Two scalar summaries are extracted from $\bar{\lambda}$:

$$\ell_1 = \|\bar{\lambda}\|_1 = \sum_{r=1}^R |\bar{\lambda}(t_r)|, \quad \ell_2 = \|\bar{\lambda}\|_2 = \sqrt{\sum_{r=1}^R \bar{\lambda}(t_r)^2}. \quad (8)$$

The L^1 norm captures the total “area” of the averaged landscape — large when many bars overlap in the same region of the filtration axis. The L^2 norm is closer to a peak detector — large when one or two regions dominate. Together they summarise the dominance hierarchy of topological features while remaining computationally cheap (no per-level vectorisation is stored).

4.4 Summary: 8 Features per Diagram

Table 2: Scalar features extracted from each persistence diagram.

Index	Feature	Equation	Interpretation
1	carl_f1	(1)	Birth-weighted total persistence (signed)
2	carl_f2	(2)	Death-distance-weighted total persistence
3	carl_f3	(3)	$b^2 \times p^4$: long bars at extreme births
4	carl_f4	(4)	$(d_{\max} - d)^2 \times p^4$: long bars dying early
5	carl_f5_max	(5)	Most persistent bar (single max)
6	entropy	(6)	Complexity / spread of persistence values
7	landscape_l1	(8)	L^1 norm of mean persistence landscape
8	landscape_l2	(8)	L^2 norm of mean persistence landscape

Applying these 8 features to each of the 3 diagrams yields $3 \times 8 = 24$ raw topological features per time series. After mutual information-based feature selection (Section 5), 18 of these are retained.

The full feature naming convention is:

$$\text{topo_}\{\text{diagram}\}_\{\text{feature}\}$$

where $\text{diagram} \in \{\text{sub_H0}, \text{sup_H0}, \text{H1}\}$ and $\text{feature} \in \{\text{carl_f1}, \text{carl_f2}, \text{carl_f3}, \text{carl_f4}, \text{carl_f5_max}, \text{entropy}, \text{landscape_l1}, \text{landscape_l2}\}$.

5 Feature Selection

Feature selection is performed using *mutual information* (MI), a non-parametric measure of statistical dependence:

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x) p(y)}. \quad (9)$$

MI equals zero if and only if X and Y are statistically independent, and is invariant under monotone transformations. It captures both linear and non-linear dependencies, making it more general than Pearson correlation for the highly non-Gaussian distribution of persistence-based features.

Leakage prevention. MI is estimated on the *training split only* and then applied to select features before any model sees the test data. Computing MI on the full dataset would leak test-label information into feature selection and inflate reported accuracy (Kaufman et al., 2012). In practice, MI is estimated using the k -nearest-neighbour estimator of Kraskov et al. (2004) as implemented in `sklearn.feature_selection.mutual_info_classif` (Pedregosa et al., 2011).

Selection result. Of the 24 raw topological features, 18 are retained on the 10-class training split. The 6 dropped features are `carl_f5_max` and `landscape_12` from each of the three diagrams (`H1`, `sub_H0`, `sup_H0`).

Why high-MI features are dropped. It would be incorrect to attribute these removals solely to low MI: in fact, `sup_H0__carl_f5_max` and `sup_H0__landscape_12` carry some of the *highest* MI scores in the pool. The selection procedure combines MI ranking with a *correlation-based redundancy filter*: among groups of features whose pairwise Pearson correlation exceeds a threshold, only one representative is kept. The six dropped features fall into two structurally redundant pairs per diagram:

- `carl_f5_max` (maximum bar lifetime) is highly correlated with `landscape_11` (area under the mean landscape), because both grow monotonically with the length of the dominant persistent bar;
- `landscape_12` (L^2 norm of the mean landscape) is highly correlated with `landscape_11` (L^1 norm of the same vector) since both are norms of the same discrete function.

The retained 18 features therefore represent the same topological information in a less redundant form, rather than a strictly higher-MI subset.

6 Literature Review: TDA for Time Series

6.1 Foundations

Persistent homology was formalised in a computational context by Edelsbrunner et al. (2002) and Zomorodian and Carlsson (2005), who introduced efficient algorithms for its computation. The stability theorem — critical for applications to noisy data — was proved by Cohen-Steiner et al. (2007). Carlsson (2009) provided the first broad survey of TDA as a practical data analysis methodology, establishing the language and goals that shaped subsequent applied work.

6.2 Vectorisation Methods

Converting persistence diagrams to machine-learning-compatible vectors has been an active research area. Bubenik (2015) introduced persistence landscapes, establishing their statistical properties (mean, variance, central limit theorem) in a Hilbert space setting. Adcock et al. (2016) characterised the ring of polynomial invariants of persistence diagrams, motivating the Carlsso coordinates used in this work. Chintakunta et al. (2015) proposed persistent entropy as a single-scalar complexity measure. More recently, persistence images (Adams et al., 2017) and the sliced Wasserstein kernel (Carrière et al., 2017) have extended the vectorisation toolkit; however, for the present application the scalar features described above are computationally efficient and interpretable.

6.3 TDA Applied to Time Series

Perea and Harer (2015) introduced the sliding window embedding (a variant of Takens embedding) and proved that it produces circles in the delay space for periodic signals, providing a rigorous foundation for using H_1 features to detect periodicity. The period manifests as a highly persistent H_1 class with $\text{pers} \approx (\text{signal amplitude})$.

Umeda (2017) applied sublevel and superlevel filtrations directly to financial and physiological time series, showing that persistence diagrams discriminate between different regimes more robustly than classical spectral features. Gidea and Katz (2018) demonstrated that the L^1 norm of the first persistence landscape detects early warning signals before financial crashes — a structural break detection task directly analogous to the `trend_shift` and `variance_shift` classes studied here.

Seversky et al. (2016) performed a systematic comparison of TDA-based features against classical time series descriptors on UCR classification benchmarks and found that sublevel persistence features compete with or outperform dynamic time warping and shapelet methods on datasets with irregular structural transitions, while underperforming on noise-heavy or purely stochastic datasets.

More recent work has pushed TDA for time series in directions that are directly relevant to the present pipeline. Lin (2022) proposed a *temporal filtration* that augments standard persistence with explicit time ordering information, addressing a key limitation of purely value-based sublevel and superlevel constructions. Elyaagoubi et al. (2023) extended the discussion from univariate trajectories to multivariate time series, especially brain-signal settings where both channel-wise evolution and cross-channel structure matter. Ferrà et al. (2023) combined persistence landscapes of time series with neural networks and an attribution layer, showing that topological summaries can be both predictive and interpretable in supervised classification settings.

Bridging the gap between theoretical tools and industrial anomaly detection, Pérez-Fernández et al. (2023) applied TDA to multivariate sensor time series from manufacturing processes, showing that persistence-based features detect hidden structural patterns that remain invisible to classical multivariate statistics. This is especially relevant to the anomaly classes (`collective_anomaly`, `point_anomaly`, `variance_shift`) studied in the present work.

Application-oriented studies from 2024 further support the usefulness of TDA for complex sequential data. Flammer (2024) used persistent homology for classification of chaotic multivariate EEG windows and reported that topology is effective for

detecting seizure-related structure after suitable dimension reduction. Zheng et al. (2024) proposed a multichannel EEG pipeline that couples Hilbert–Huang signal representations with topological features, illustrating how TDA can be integrated with more classical signal-processing front ends rather than used in isolation.

Chazal and Michel (2021) provides a comprehensive modern introduction to TDA with statistical perspectives.

6.4 TDA and Machine Learning

The combination of TDA features with standard machine learning classifiers has been explored widely in recent years. Carlsson (2020) provides an authoritative review of topological pattern recognition for point cloud data, describing the theoretical framework within which persistence-based classifiers operate. At the applied level, Hensel et al. (2021) surveyed machine learning applications of TDA across domains, noting that the combination of persistence-based features with gradient-boosted trees is the most consistently competitive approach, with random forests close behind — consistent with the CatBoost and Random Forest results reported in Section 7.

Tauzin et al. (2021) describe the `giotto-tda` Python library, which implements the persistent homology pipeline used in this work, including Vietoris–Rips computation for Takens embeddings and sublevel/superlevel filtrations for scalar functions. The underlying computation of Vietoris–Rips persistent homology is performed by the RIPSER algorithm (Bauer, 2021), one of the fastest available implementations; a Python interface is provided by Tralie et al. (2018).

For broader context, Su et al. (2026) review recent developments in TDA and topological deep learning beyond classical persistent homology. Although their scope is wider than time series alone, the review is useful for situating modern sequence-analysis pipelines within the rapidly expanding ecosystem of topological representations, vectorisations, and learning architectures.

7 Empirical Performance Summary

The topological features were evaluated on a 10-class synthetic time series classification task (`topo-test` dataset: 1,000 series, 100 per class, length 1,000 time points). Results are compared against a 100-feature TSFresh pipeline and a 118-feature hybrid concatenation. All experiments use an 80/20 stratified train/test split with seed 42. Table 3 summarises test accuracy for four classifiers across all three feature pipelines.

Table 3: Test accuracy on the 10-class topo-test benchmark.

Pipeline	Features	RF	XGBoost	CatBoost	SVM
TSFresh	100	93.0%	94.5%	95.0%	89.0%
Topology	18	85.5%	85.0%	86.5%	78.5%
Hybrid	118	92.5%	94.5%	94.5%	91.0%

Feature efficiency. The topological pipeline delivers 86.5% CatBoost accuracy from just 18 features — using $100/18 \approx 5.6\times$ fewer features than the TSFresh

pipeline at the cost of an 8.5 percentage point accuracy reduction. Expressed as accuracy per feature:

$$\begin{aligned}\text{TSFresh: } & 95.0\%/100 = 0.95\% \text{ per feature,} \\ \text{Topology: } & 86.5\%/18 = 4.81\% \text{ per feature.}\end{aligned}$$

The accuracy-per-feature ratio is therefore $4.81/0.95 \approx 5.1\times$ — topological features achieve approximately **5 $\times$ better accuracy-per-feature efficiency** than the statistical baseline.

Per-class highlights. Table 4 reports CatBoost F1 scores by class across all three pipelines.

Table 4: CatBoost test F1 scores by class across the three feature pipelines. Classes where topology matches or exceeds TSFresh are marked ($\dagger$).

Class	TSFresh	Topology	Hybrid
stationary	0.884	0.651	0.889
deterministic_trend	0.974	0.872	0.974
stochastic_trend	1.000	0.895	0.974
volatility	0.865	0.750	0.889
collective_anomaly	0.976	0.732	0.947
contextual_anomaly	1.000	0.974	1.000
mean_shift	0.909	0.850	0.870
point_anomaly	1.000	1.000 $\dagger$	1.000 $\dagger$
trend_shift	0.919	0.974$\dagger$	0.919
variance_shift	0.974	0.976$\dagger$	1.000 $\dagger$
Macro F1	0.950	0.867	0.946

Topology *surpasses* TSFresh on **trend_shift** (0.974 vs. 0.919) and matches it on **variance_shift** and **point_anomaly**. Both of these advantages are structurally interpretable:

- **Trend shift:** A slope change creates a distinctive asymmetry in the *sublevel* filtration: the segment on one side of the change point has a different minimum than the other, and the two sublevel components merge at a filtration level determined by the change-point amplitude. The sublevel persistence diagram therefore contains a single high-persistence point whose birth captures the lower of the two minima, which is exactly the pattern that `sub_H0_car1_f3` emphasises (large $b^2 \cdot p^4$ at extreme births). This is consistent with the empirical finding that `sub_H0_car1_f3` is the single most important feature in the hybrid model. TSFresh chunk linear-trend features miss this signature whenever the change point falls between chunks.
- **Variance shift:** A step increase in variance produces a visible change in the density of sublevel crossings. Carlsson coordinates and persistent entropy in the sublevel diagram encode this directly. TSFresh change-quantile features capture similar information but are less robust to the exact location of the shift.

- **Stationary (worst case):** TSFresh has a decisive advantage because autocorrelation features, spectral flatness, and stationarity-related statistics directly encode the absence of a trend or structural change. Persistence diagrams, being invariant to temporal ordering, cannot detect stationarity without supplementary temporal information.

Most important topological feature. In the hybrid CatBoost model (118 features: 100 TSFresh + 18 topological), `topo__sub_H0__car1_f3` is the *single most important feature* with an importance score of 13.2%, outranking all 100 TSFresh features. This coordinate (Equation 3) takes the form

$$f_3(D) = \sum_i b_i^2 \cdot (d_i - b_i)^4,$$

combining a quadratic birth weight with a quartic persistence weight. The b_i^2 factor amplifies bars born at extreme filtration values, while p_i^4 aggressively suppresses short (noise) bars. The result is a single scalar that fires strongly when a series contains *long-lived bars whose births occur at extreme signal levels* — exactly the pattern produced by trend-like asymmetric global minima and persistent secondary minima of anomaly classes. Stationary processes, by contrast, produce many short-lived low-persistence components with births clustered near zero, so the $b^2 \cdot p^4$ weighting drives f_3 toward small values. This is why `sub_H0__car1_f3` is highly discriminative across the ten classes studied here.

Practical limitations of the topological pipeline

For balanced use of topological features it is worth being explicit about their limitations, several of which have been noted in the literature:

1. **Temporal order invariance.** As discussed in Section 3, sublevel and superlevel diagrams are invariant to permutations of the time axis. This means that autocorrelation, stationarity, and volatility clustering — all of which depend on temporal ordering — are not directly representable. The Takens embedding (H_1) partially restores temporal structure, but only for the delay parameter τ chosen.
2. **Computational cost.** Computing Vietoris–Rips persistent homology scales poorly with the number of points. For a time series of length T with embedding dimension d , the Takens point cloud has $T - (d - 1)\tau$ points, and Rips computation is $O(n^3)$ in the worst case. In practice, the RIPSER algorithm (Bauer, 2021) dramatically reduces this cost via matrix reduction, and sublevel/superlevel filtrations remain $O(T \log T)$.
3. **Hyperparameter sensitivity.** The Takens embedding requires choosing τ (delay) and d (embedding dimension). A poorly chosen τ may produce a degenerate point cloud. In practice, the false nearest neighbours method or mutual information criterion is used to select τ (Perea & Harer, 2015; Takens, 1981).

4. **Univariate focus.** The filtrations described here are designed for univariate series. Extending them to multivariate time series requires additional choices (e.g., channel-wise vs. joint embeddings), an active research area (Elyaagoubi et al., 2023; Zheng et al., 2024).

8 Conclusion

Topological Data Analysis provides a principled, mathematically grounded approach to time series feature extraction that is complementary to classical statistical methods. The persistent homology framework, applied through sublevel, superlevel, and Takens delay-embedding filtrations, extracts 18 scalar features that:

1. Achieve 86.5% classification accuracy on a 10-class synthetic benchmark with only 18 features ($\sim 5.1\times$ better accuracy-per-feature than the 100-feature TSFresh pipeline);
2. Outperform TSFresh on structural break detection tasks (`trend_shift`, `variance_shift`);
3. Contribute the single most important feature in a 118-feature hybrid model (`sub_H0__car1_f3`, importance 13.2%);
4. Provide stable, theoretically well-founded representations (Stability Theorem) that are robust to small measurement perturbations.

The primary limitation of topological features is their insensitivity to temporal ordering: persistence diagrams are invariant to permutation of time steps, which makes them poor descriptors for stationarity, autocorrelation structure, and volatility clustering — properties that TSFresh captures well. The hybrid pipeline exploits both strengths and is recommended when both structural change detection and temporal regularity characterisation are required.

References

- Adams, H., Emerson, T., Kirby, M., Neville, R., Peterson, C., Shipman, P., Chepushtanova, S., Hanson, E., Motta, F., & Ziegelmeier, L. (2017). Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8), 1–35.
- Adcock, A., Carlsson, E., & Carlsson, G. (2016). The ring of algebraic functions on persistence bar codes. *Homology, Homotopy and Applications*, 18(1), 381–402. <https://doi.org/10.4310/HHA.2016.v18.n1.a21>
- Bauer, U. (2021). Ripser: Efficient computation of Vietoris–Rips persistence barcodes. *Journal of Applied and Computational Topology*, 5(3), 391–423. <https://doi.org/10.1007/s41468-021-00071-5>
- Bubenik, P. (2015). Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16, 77–102.
- Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308. <https://doi.org/10.1090/S0273-0979-09-01249-X>
- Carlsson, G. (2020). Topological pattern recognition for point cloud data. *Acta Numerica*, 29, 1–70. <https://doi.org/10.1017/S0962492914000051>
- Carrière, M., Cuturi, M., & Oudot, S. (2017). Sliced wasserstein kernel for persistence diagrams. *Proceedings of the 34th International Conference on Machine Learning*, 664–673.
- Chazal, F., & Michel, B. (2021). An introduction to topological data analysis: Fundamental and practical aspects for data scientists. *Frontiers in Artificial Intelligence*, 4, 667963. <https://doi.org/10.3389/frai.2021.667963>
- Chintakunta, H., Gentimis, T., Gonzalez-Diaz, R., Jimenez, M.-J., & Krim, H. (2015). An entropy-based persistence barcode. *Pattern Recognition*, 48(2), 391–401. <https://doi.org/10.1016/j.patcog.2014.06.023>
- Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package). *Neurocomputing*, 307, 72–77. <https://doi.org/10.1016/j.neucom.2018.03.067>
- Cohen-Steiner, D., Edelsbrunner, H., & Harer, J. (2007). Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1), 103–120. <https://doi.org/10.1007/s00454-006-1276-5>
- Edelsbrunner, H., & Harer, J. (2010). *Computational topology: An introduction*. American Mathematical Society. <https://doi.org/10.1090/mbk/069>
- Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2002). Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4), 511–533. <https://doi.org/10.1007/s00454-002-2885-2>
- Elyaagoubi, S. E., Ford, C., & Maroulas, V. (2023). Topological data analysis for multivariate time series data. *Entropy*, 25(11), 1509. <https://doi.org/10.3390/e25111509>
- Ferrà, A., Casacuberta, C., & Pujol, O. (2023). Importance attribution in neural networks by means of persistence landscapes of time series. *Neural Computing and Applications*, 35, 20143–20156. <https://doi.org/10.1007/s00521-023-08731-6>
- Flammer, M. (2024). Persistent homology-based classification of chaotic multi-variate time series: Application to electroencephalograms. *SN Computer Science*, 5, 107. <https://doi.org/10.1007/s42979-023-02396-7>

- Gidea, M., & Katz, Y. (2018). Topological data analysis of financial time series: Landscapes of crashes. *Physica A: Statistical Mechanics and its Applications*, 491, 820–834. <https://doi.org/10.1016/j.physa.2017.09.028>
- Hatcher, A. (2002). *Algebraic topology*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511662584>
- Hensel, F., Moor, M., & Rieck, B. (2021). A survey of topological machine learning methods. *Frontiers in Artificial Intelligence*, 4, 681108. <https://doi.org/10.3389/frai.2021.681108>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 15. <https://doi.org/10.1145/2020408.2020496>
- Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6), 066138. <https://doi.org/10.1103/PhysRevE.69.066138>
- Lin, B. (2022). Topological data analysis in time series: Temporal filtration and application to single-cell genomics. *Algorithms*, 15(10), 371. <https://doi.org/10.3390/a15100371>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, È. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Perea, J. A., & Harer, J. (2015). Sliding windows and persistence: An application of topological methods to signal analysis. *Foundations of Computational Mathematics*, 15(3), 799–838. <https://doi.org/10.1007/s10208-014-9206-z>
- Pérez-Fernández, A., Gallego-Lavilla, J., González-Marcos, A., & Alba-Elias, F. (2023). Topological data analysis for detecting hidden patterns in multivariate time series. *Neurocomputing*, 535, 142–152. <https://doi.org/10.1016/j.neucom.2023.03.031>
- Seversky, L. M., Davis, S., & Berger, M. (2016). On time-series topological data analysis: New data and opportunities. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 1014–1022. <https://doi.org/10.1109/CVPRW.2016.131>
- Su, Z., Liu, X., Bou Hamdan, L., Maroulas, V., Wu, J., Carlsson, G., & Wei, G.-W. (2026). Topological data analysis and topological deep learning beyond persistent homology: A review. *Artificial Intelligence Review*, 59, 58. <https://doi.org/10.1007/s10462-025-11462-w>
- Takens, F. (1981). Detecting strange attractors in turbulence. *Lecture Notes in Mathematics*, 898, 366–381. <https://doi.org/10.1007/BFb0091924>
- Tauzin, G., Lupo, U., Tunstall, L., Perea, J. A., Caorsi, M., Medina-Mardones, A., Dassatti, A., & Hess, K. (2021). Giotto-tda: A topological data analysis toolkit for machine learning and data exploration. *Journal of Machine Learning Research*, 22(39), 1–6.
- Tralie, C., Saul, N., & Bar-On, R. (2018). Ripser.py: A lean persistent homology library for python. *Journal of Open Source Software*, 3(29), 925. <https://doi.org/10.21105/joss.00925>

- Umeda, Y. (2017). Time series classification via topological data analysis. *Information and Media Technologies*, 12(2), 228–239. <https://doi.org/10.11185/imt.12.228>
- Zheng, J., Feng, Z., & Ekstrom, A. D. (2024). Towards analysis of multivariate time series using topological data analysis. *Mathematics*, 12(11), 1727. <https://doi.org/10.3390/math12111727>
- Zomorodian, A., & Carlsson, G. (2005). Computing persistent homology. *Discrete & Computational Geometry*, 33(2), 249–274. <https://doi.org/10.1007/s00454-004-1146-y>